```
_erbout.<<(( number ).to_s);
_erbout.<< " bottles of beer...\n".freeze;
end;
_erbout.<< "\n".freeze;
_erbout
```

Notice how ERB builds a string, _erbout, that contains both the static strings from the template and the results of executing expressions (in this case the value of number).

Embedding ERB in Your Code

So far we've seen ERB running as a command-line filter. However, the most common use is to use it as a library in your own code. Many Ruby web frameworks automatically use ERB templates for output.

```
require "erb"

SOURCE =
%{<% (min..max).each do |number| %>
The number is <%= number %>
<% end %>
}

erb = ERB.new(SOURCE)

min = 4
max = 6
puts erb.result(binding)
```

produces:

```
The number is 4

The number is 5

The number is 6
```

Notice how we can use local variables within the ERB template. This works because we pass the current *binding* to the result method. ERB can use this binding to make it look as if the template is being evaluated in the context of the calling code. Using bindings is sometimes awkward, especially if you want to limit what values are available to the template. ERB also provides result_with_hash, which, true to its name, takes a hash argument, and within the template, variable names are resolved as keys to the hash.

Be aware that in the standard version of ERB, you can't use the -%> trick to suppress blank lines. (That's why we have the extra blank lines in the output in the previous example.) You can specify the way ERB handles blanks by passing a trim_mode: keyword argument to ERB.new

The values that trim_mode can take are: * %: Allows you to have lines starting with % processed as Ruby coder * >: Lines ending in %> don't have a newline added to the result. * <>: Same as above, but for lines starting with <% and ending with %> * -: Blank lines ending in -%> are omitted. (This is the setting used in Rails.)

ERB comes with excellent documentation.[2]

2. https://docs.ruby-lang.org/en/master/ERB.html